

Autonomous agents and Intelligent Robotics

Giorgio Battistelli

Department of Information Engineering, University of Florence, Italy



UNIVERSITÀ
DEGLI STUDI
FIRENZE

DINFO
DIPARTIMENTO DI
INGEGNERIA
DELL'INFORMAZIONE

2 Robot motion planning

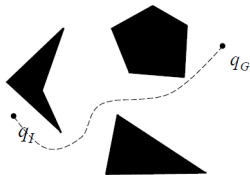
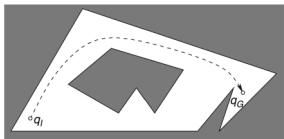
2.1 **Robot motion planning:** Configuration space and motion planning

Robot motion planning

Robot motion planning: given

- a *start* configuration (pose) of the robot
- a desired *goal* configuration
- a geometric description of the robot and the environment

Find a collision-free path from the initial to the goal configuration, possibly minimizing a cost such as path length.



Configuration space

- **Robot configuration:** a set of generalized coordinates that uniquely determines the pose of every point of the robot
- **Configuration space $\mathcal{C}$:** set of all the possible configurations of a robot

- Usually configuration is expressed as a vector q of **positions** and **orientations**
- For a point-like robot moving in 2D

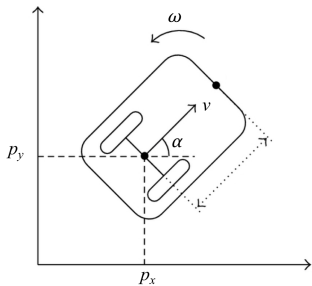
$$q = \begin{bmatrix} p_x \\ p_y \end{bmatrix} \quad \mathcal{C} = \mathbb{R}^2$$

- For a point-like robot moving in 3D

$$q = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix} \quad \mathcal{C} = \mathbb{R}^3$$

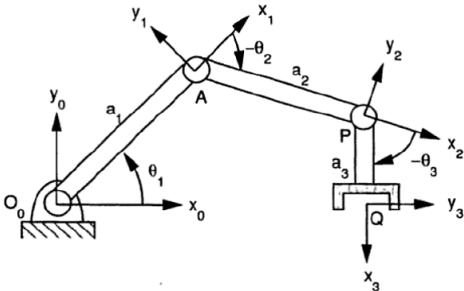
- Real robots are not point-like: the configuration space can be **high-dimensional** and its topology can be **non-cartesian**

Configuration space



Unicycle

$$q = \begin{bmatrix} p_x \\ p_y \\ \theta \end{bmatrix}$$



Planar manipulator

$$q = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix}$$

Configuration space of the unicycle

- For the unicycle,

$$q = \begin{bmatrix} p_x \\ p_y \\ \theta \end{bmatrix}$$

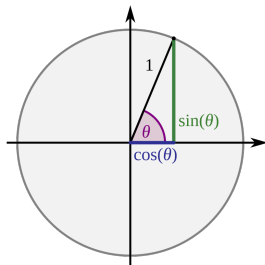


- The orientation θ specifies a rotation in $2D$
- Configuration space for the unicycle

$$\mathcal{C} = \mathbb{R}^2 \times SO(2)$$

where $SO(2)$ is the **special orthogonal group** of 2D rotations

- $SO(2)$ is isomorphic to the unit circle

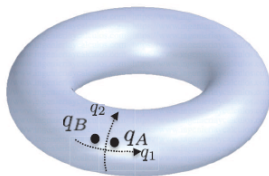
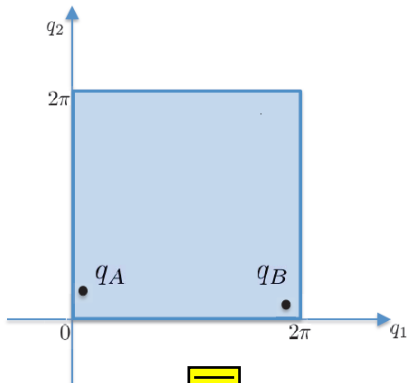


Configuration space of a planar manipulator

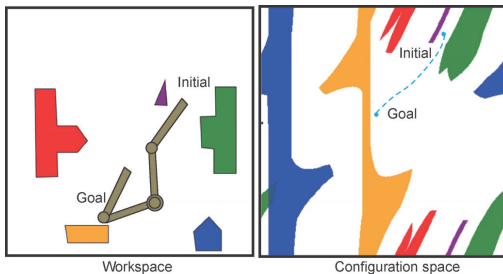
- For a planar manipulator with 2 degrees of freedom

$$q = \begin{bmatrix} q_1 \\ q_2 \end{bmatrix} = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}$$

- Each degree of freedom specifies a rotation in 2D
- The configuration space is $\mathcal{C} = SO(2) \times SO(2)$ which is isomorphic to a **torus**

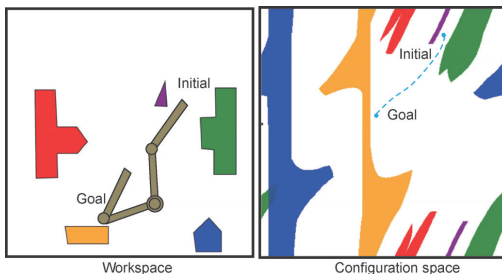


Workspace vs configuration space



- **Workspace $\mathcal{W}$:** physical, **real-world** space in which the robot operates
 - 2D or 3D cartesian space
 - useful for defining global constraints on robot motion such as the boundaries of the environment and the location of obstacles
- **Configuration space $\mathcal{C}$:** **abstract** space of all possible robot configurations
 - each point in the configuration space corresponds to a unique robot configuration, defining positions and orientations of all robot joints and/or degrees of freedom
 - dimension equal to the robot degrees of freedom, typically non-Cartesian
 - primarily used for motion planning and collision avoidance

Free space and obstacle space



- $\mathcal{W}_{\text{obs},k} \subset \mathcal{W}$ points of the workspace occupied by the k -th obstacle
- $\mathcal{R}(q) \subset \mathcal{W}$ points of the workspace occupied by a robot in configuration q
- $\mathcal{C}_{\text{obs},k} = \{q \in \mathcal{C} : \mathcal{W}_{\text{obs},k} \cap \mathcal{R}(q) \neq \emptyset\}$ set of configuration in which the robot is in collision with the k -th obstacle
- **Free space** $\mathcal{C}_{\text{free}}$ set of configurations that avoid collision with the obstacles

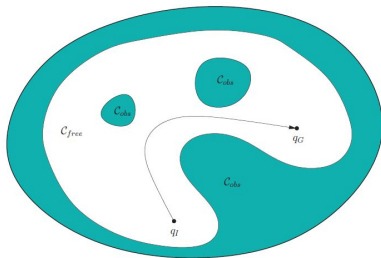
$$\mathcal{C}_{\text{free}} = \mathcal{C} \setminus \mathcal{C}_{\text{obs}}$$

with $\mathcal{C}_{\text{obs}} = \bigcup_k \mathcal{C}_{\text{obs},k}$ the **obstacle space**

Robot motion planning

- **Configuration space** $\mathcal{C}$
- **Free space** $\mathcal{C}_{\text{free}}$ set of configurations that avoid collision with the obstacles
- **Obstacle space** $\mathcal{C}_{\text{obs}} = \mathcal{C} \setminus \mathcal{C}_{\text{free}}$
- **Start** q_I initial robot configuration
- **Goal** q_G desired robot configuration

Robot motion planning: Given $\mathcal{C}$, $\mathcal{C}_{\text{free}}$ and two points $q_I, q_G \in \mathcal{C}_{\text{free}}$, automatically compute a path in $\mathcal{C}_{\text{free}}$ from q_I to q_G



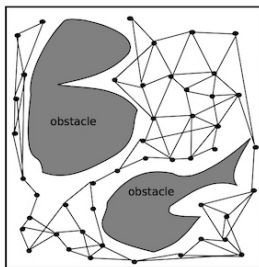
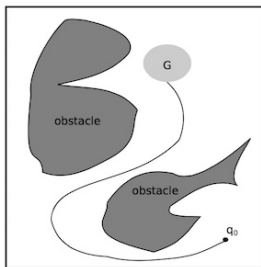
- Ideally we want the shortest path
- q_G can be replaced by a set $\mathcal{C}_G$ of desired configurations

2.2 **Robot motion planning:** Planning-based solution

Planning-based solution

Robot motion planning: Given $\mathcal{C}, \mathcal{C}_{\text{free}}$ and two points $q_I, q_G \in \mathcal{C}_{\text{free}}$, automatically compute a path in $\mathcal{C}_{\text{free}}$ from q_I to q_G

- Planning-based solutions usually follow these steps:
 - 1 Build a graph representing a **road map** in the free space
 - 2 Find the **optimal path** from start to goal in the road map (value iteration, Dijkstra, A^* , D^*)
 - 3 Transform the discrete path into a **smooth trajectory** for the robot



Road map

Road map: a graph in which

- each node is a configuration in the robot state space
- each edge is a collision-free path in the robot state space



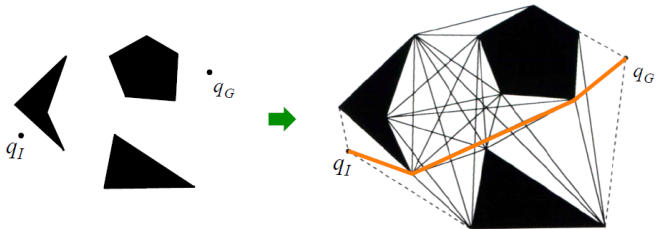
- Approaches for building the road map:
 - **geometric** road map (for example *visibility graph*)
 - **cell decomposition** (*exact* or *approximate*)
 - **sampling-based** methods (for example *probabilistic road map*, *rapidly-exploring random trees*)
- For simplicity, we describe these methods for a *point-like robot* moving in the plane $q(t) \in \mathbb{R}^2$

Visibility graph

Visibility graph: a graph in which

- the nodes are the start, the goal, and all the *vertices* of the obstacles
- two nodes are connected when the segment joining them does *not* cross any obstacle

- We assume a **polygonal** world
- The optimal path is a *polygonal* line connecting start and goal through vertices of the obstacles



Visibility graph: pros and cons

Advantages

- **Complete:** if a collision-free path exists, the algorithm is guaranteed to find it, if no path exists, the algorithm correctly reports failure
- Yields the **shortest path** in polygonal environments
- Relatively simple to implement in 2D with polygonal obstacles
- Compact representation: only obstacle vertices and start/goal as nodes

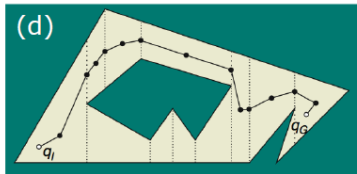
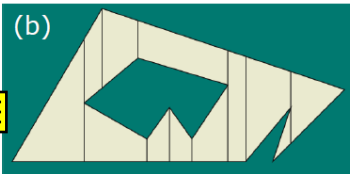
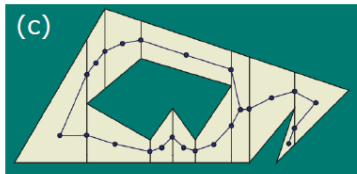
Drawbacks

- **Requires polygonal world assumption** (not general for arbitrary obstacles) or polygonal approximation
- **High computational complexity in dense environments**, $O(N_v^2)$ edges for N_v vertices
- **Path runs very close to obstacles** $\Rightarrow$ poor robustness to modeling/sensing errors
- Not easily extendable to high-dimensional configuration spaces

Exact cell decomposition

Exact cell decomposition:

- decompose the obstacle-free space into **non-overlapping** cells
- construct connectivity graph to represent **adjacencies**



Exact cell decomposition

Trapezoid cell decomposition:

- ➊ Decompose the free space into **trapezoids** with vertical side segments by shooting rays upward and downward from each polygon vertex
 - ➋ Place one node in the interior of every trapezoid, pick e.g. the centroid
 - ➌ Place one node in every vertical segment
 - ➍ Connect the nodes
- We assume a **polygonal** world
 - Complexity in the order of $N_v \log N_v$ where N_v is the number of polygon vertices
 - Exact decomposition methods can be involved and inefficient for complex problems

Exact cell decomposition: pros and cons

Advantages

- Complete
- Lower computational complexity than visibility graph, $O(N_v \log N_v)$ with N_v obstacle vertices
- Provides a structured representation of the free space as a connectivity graph of cells

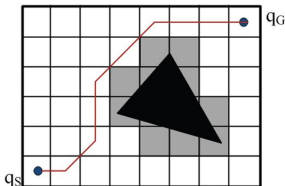
Drawbacks

- Applicable only to polygonal environments
- Limited to 2D worlds (not easily extendable to higher-dimensional $\mathcal{C}$ -spaces)

Approximate cell decomposition

Approximate cell decomposition: uses cells with the same *simple predefined shape*, for example *occupancy grids*, *quadtree decomposition*

- **Occupancy grids** represent the environment as a grid in which each cell can be either occupied by the robot, free, or occupied by an obstacle
- Decompose the configuration space $\mathcal{C}$ in cells and mark as occupied all cells that are not contained in the free space $\mathcal{C}_{\text{free}}$
- Approximate cell decomposition is simpler to implement and more numerical stable
- Accuracy can be improved using an *adaptive* grid (finer grid near the obstacles)

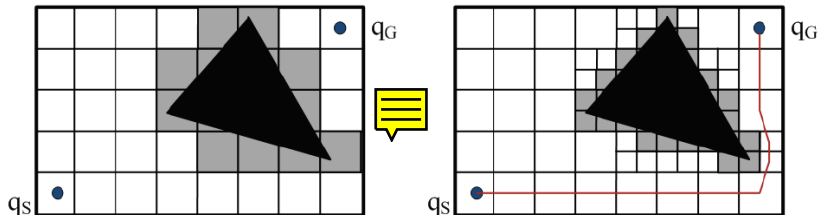


Approximate cell decomposition

Difficulty: cell decomposition with fixed cells is not complete (in some cases it cannot find a path even if it exists)

Solution: use an *adaptive* grid (finer grid near the obstacles)

- **Distinguish between:**
 - **Fully occupied cells:** cells entirely contained in the obstacle space $\mathcal{C}_{obs}$
 - **Mixed cells:** cells that only partially intersect the obstacle space $\mathcal{C}_{obs}$
- Try to find a path using the current set of cells. If no path is found subdivide the MIXED cells and try again with the new set of cells



Approximate cell decomposition: pros and cons

Advantages

- Easier to implement than exact cell decomposition
- Can handle also non-polygonal obstacles
- Accuracy can be improved with adaptive grids (finer near obstacles)
- **Fixed grid: resolution complete**
If a path exists representable at the chosen grid resolution, the algorithm is guaranteed to find it
- **Adaptive grid: semi-complete**
If a path exists, the algorithm is guaranteed to find it in finite time



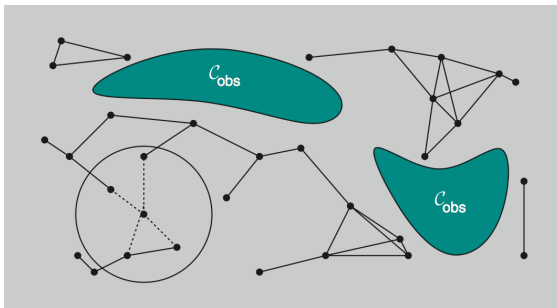
Drawbacks

- **Not complete**
- **Curse of dimensionality:** computational complexity grows exponentially with the configuration space dimension: $O(K^{n_c})$ cells, with K the resolution per degree of freedom and n_c the dimension of $\mathcal{C}$ -space

Probabilistic road map

Probabilistic road map (PRM): main ideas

- take **random samples** in the configuration space,
- declare them as nodes if they are in the free space
- try to connect nearby nodes with **local planner**



Probabilistic road map

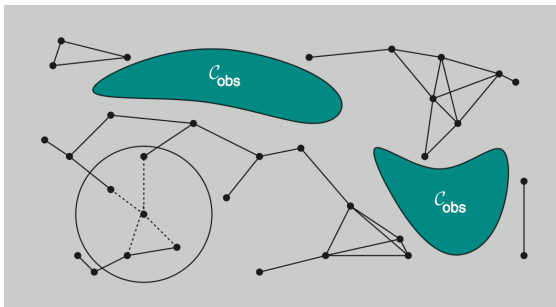
PRM algorithm:

Repeat

- 1 **Repeat** randomly generate a point q_{rand} in the configuration space $\mathcal{C}$
Until q_{rand} is in the free space $\mathcal{C}_{\text{free}}$
 - 2 Add q_{rand} as node of the road map
 - 3 Find all *nearby* nodes Q_{near}
 - 4 **For** any $q_{\text{near}} \in Q_{\text{near}}$
 If the local planner can find a collision-free path between q_{rand} and q_{near}
 add an edge $\{q_{\text{rand}}, q_{\text{near}}\}$
- Until** the road map is dense enough

Probabilistic road map

- **Nearby nodes:** are defined using either k -nearest neighbors or ε -neighborhood
- **ε -neighborhood:** an existing node q belongs to Q_{near} $\Leftrightarrow$ the distance between q and q_{rand} is less than a threshold ε
 - the choice of the *hyperparameter* ε is critical
- **k -nearest neighbors:** Q_{near} consists of the k existing nodes with smallest distance from q_{rand}
 - the resulting graph can have also *long-distance* connections
 - choice of k less difficult than the choice of ε



Probabilistic road map

- Implementation of PRM requires: collision detection and local planner
- **Collision detection:** an algorithm to check whether a given configuration q is in collision with the obstacles or instead it belongs to the free space $\mathcal{C}_{\text{free}}$
 - the complexity of collision detection depends on the geometry of the robot and the obstacles
 - collision detection usually takes the largest amount of time in motion planning
- **Local planner:** an algorithm to find a collision-free path between q_{rand} and q_{near}
 - easiest solution: consider the line-of-sight segment and check that it is collision free
 - as an alternative advanced local planning strategies can be used (for example, *artificial potential fields*)

Question: how to check that the segment is collision free?

Trivial solution: sample the segment and only check a finite number of configurations (depend on the resolution, tradeoff between accuracy and computational complexity)

For specific problems more principled solutions can be found!

Probabilistic Roadmap (PRM): pros and cons

Advantages

- **Probabilistically complete:** if a path exists, the probability of finding it tends to 1 as the number of samples grows
- Can handle any type of obstacles (not limited to polygonal shapes)
- More efficient than cell decomposition since samples are concentrated in the free space
- Flexible: different local planners and sampling strategies can be plugged in (e.g., bias near obstacles for narrow passages)

Drawbacks

- **Not complete:** if no path exists, the algorithm may never terminate with a definitive answer
- Path quality depends on sampling density and connection strategy
- Collision checking dominates computational cost
- Performance can degrade in high-dimensional configuration spaces (**curse of dimensionality**)

Rapidly-exploring random trees



Rapidly-exploring random trees (RRT): explore the free space by incrementally building a *tree rooted at the start*

- Sampling-based (Monte Carlo) approach because the tree is incrementally built by randomly sampling in the space
- The resulting tree is a sort of stochastic fractal



45 iterations



2345 iterations

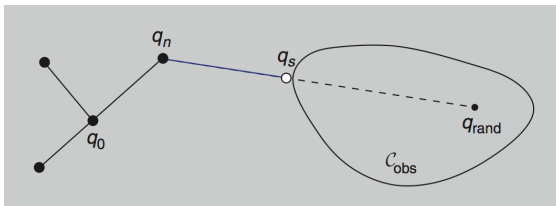
Rapidly-exploring random trees

RRT algorithm:

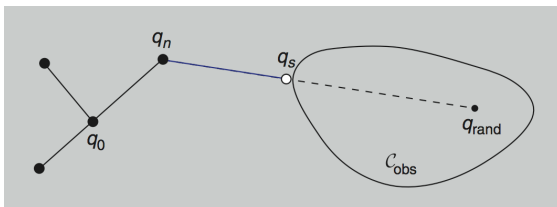
Repeat

- 1 Randomly generate a point q_{rand} in the space
- 2 Find the closest node q_{near} of the tree
- 3 Find the point q_s on the segment joining q_{rand} and q_{near} that
 - maximizes the distance from q_{near}
 - under the constraint that the segment joining q_{near} and q_s is collision-free
- 4 Add q_s as node and add an edge $\{q_{\text{near}}, q_s\}$ to the tree

Until the goal is reached or the computational budget is exhausted



Rapidly-exploring random trees



- To reduce computation q_s can be computed approximately, for example, by considering a finite set of regularly spaced candidate points on the segment joining q_{near} and q_{rand}
- We can perform motion planning by
 - at each iteration, sampling the goal with probability p_{goal}
 - then terminate the tree construction when the goal is added to the tree
- RRT is a good balance between greedy search and exploration, however rate of convergence difficult to establish

RRT variants

Bidirectional RRT

- Grow two trees, one from the start and one from the goal
- **Try to connect them** (each new node of one tree is used as target for the other)
- Often faster convergence than a single tree

RRT*



- Same exploration principle as RRT, but adds a **rewiring step**
- **Rewiring:** when a new node is added, nearby nodes are checked and possibly reconnected **if going through the new node yields a shorter path from the start**
- **Asymptotically optimal:** as the number of samples grows, **the path length converges to the optimal one**
- Higher computational cost compared to standard RRT

PRM vs RRT

	PRM	RRT
Structure	Builds a roadmap (graph) in $\mathcal{C}_{\text{free}}$	Builds an incremental tree rooted at the start
Query type	Multi-query (once built, can answer multiple start-goal queries)	Single-query (tailored to a specific start)
Completeness	Probabilistically complete	Probabilistically complete
Optimality	Optimal with a fixed ϵ -neighborhood	Not optimal (RRT*: asymptotically optimal)
Weaknesses	Performance degrades in narrow passages	Path quality often poor without RRT*
Typical use	Static environments with repeated queries	Online planning in high-dimensional spaces

Configuration space and planning

- Motion planning is naturally formulated in the **configuration space**, where each point represents a complete robot configuration
- The configuration space can be **high-dimensional** and have a **nontrivial topology** $\Rightarrow$ planning may be computationally demanding, and sampling-based methods are often preferable to explicit geometric or cell-decomposition methods

Simplified workspace planning: represent the robot by a reference point and plan its path directly in the physical workspace, typically $\mathbb{R}^2$ or $\mathbb{R}^3$.

- Obstacles can be enlarged according to the robot footprint. If orientation is ignored, the footprint can account for all admissible orientations, yielding a safe but possibly conservative approximation.
- Obstacle enlargement accounts for the robot geometry, but not necessarily for kinematic or dynamic constraints.
- For manipulators, inverse kinematics is needed to map a task-space path into configuration space; a low-level controller then tracks the resulting trajectory

Motion planning in unstructured environments

Difficulty: Robot motion planning requires a map of the environment, but usually the environment is

- **unknown** (at least partially)
 - **dynamic** (i.e. obstacles can move, appear or disappear)
 - **unstructured** (i.e. difficult to represent as a finite set of obstacles)
-
- Global motion planning typically relies on a map, which can be incrementally built or updated from sensor data
 - To deal with unknown/dynamic/unstructured environments:
 - **probabilistic graphs** (stochastic shortest path)
 - use algorithms that **avoid replanning from scratch** (incremental planners such as D* reuse previous search results when the map or edge costs change)
 - In **highly dynamic environments**: use **hybrid** methods combining motion planning with behaviors

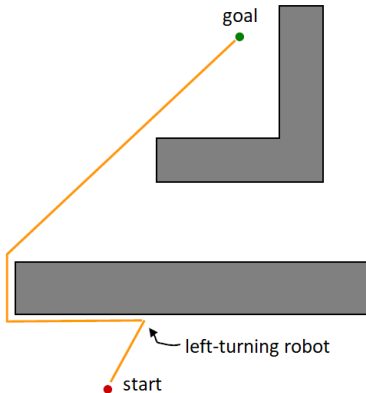
2.3 Robot motion planning: Bug algorithms



Bug algorithms

- Many planning algorithms assume **global** knowledge
- **Bug algorithms** assume only **local** knowledge of the environment and a global goal
 - known direction to goal
 - tactile sensing for obstacle detections
- Behavior-based approach based on combining two simple behaviors:
 - follow a wall (right or left)
 - move in a straight line toward goal
- Consider a point-like robot moving in a 2D environment

Bug 0 algorithm

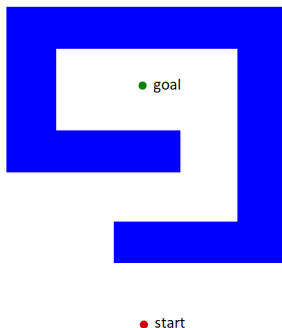


Bug 0 algorithm: alternate between two behaviors

- 1 head toward goal until the goal or an obstacle is reached
- 2 follow the obstacle until you can head toward the goal again

Bug 0 algorithm

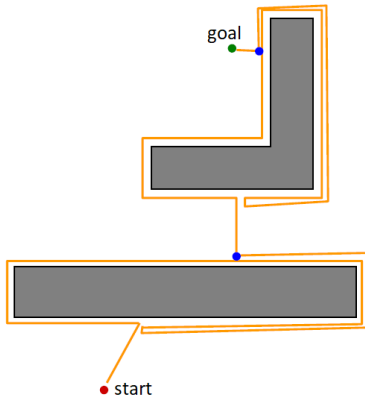
- Very simple but does not always guarantee to reach the goal when possible (i.e., it's not complete)
- The Bug 0 algorithm can be trapped in endless loops



Solution: add some memory!

Bug 1 algorithm

- Suppose that we can measure the **distance** from the goal



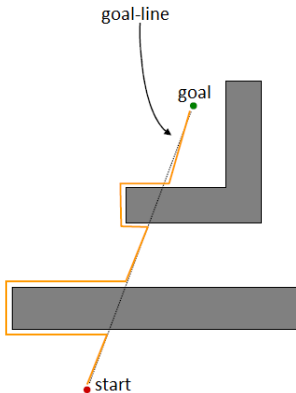
Bug 1 algorithm:

- 1 head toward goal **until** the goal or an obstacle is reached
- 2 **if** an obstacle is encountered, circumnavigate it and *remember* how close you get to the goal
- 3 return to that *closest point* (by wall-following) and go to step 1



Bug 2 algorithm

- Consider the **goal-line** going from start to goal



Bug 2 algorithm:

- 1 head toward goal on the goal-line **until** the goal or an obstacle is reached
- 2 **if** an obstacle is encountered, follow it **until** you encounter the goal-line again *closer to the goal*
- 3 leave the obstacle and go to step 1



Bug 1 vs Bug 2

- Both algorithms are complete (reach the goal when possible or otherwise terminate)
 - Bug 1 is an *exhaustive search algorithm*: it looks at all choices before deciding
 - Bug 2 is a *greedy algorithm*: it takes the first thing that looks better
- Lower bound** on the travelled distance: for both algorithms, the start-to-goal distance D

- Upper bound** on the travelled distance for Bug 1

$$D + \frac{3}{2} \sum_k P_k$$

where P_k is the perimeter of the k -th obstacle

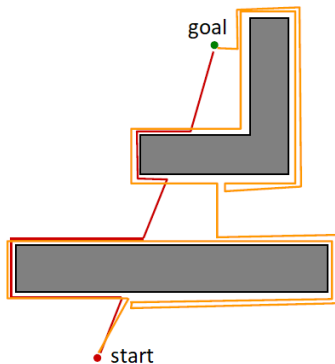
- Upper bound** on the travelled distance for Bug 2

$$D + \sum_k \frac{n_k}{2} P_k$$

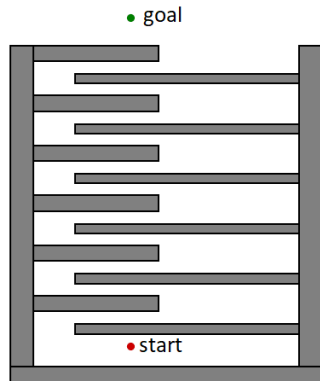
where n_k is the number of crossing points between the goal-line and the obstacle perimeter

Bug 1 vs Bug 2

- In many cases, Bug 2 will outperform Bug 1, but Bug 1 has a more predictable performance overall.



Bug 2 better than Bug 1

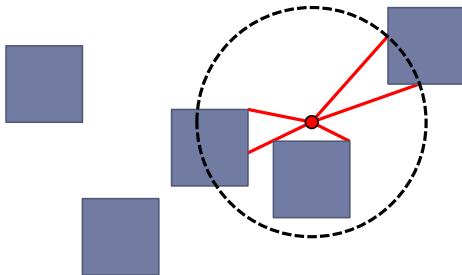


Bug 1 better than Bug 2



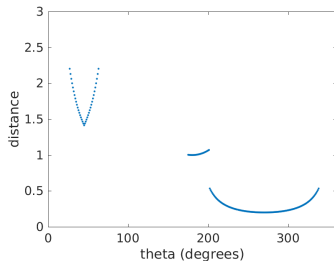
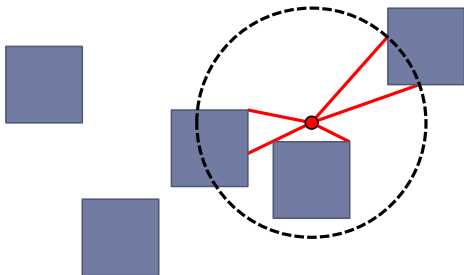
Towards better sensing

- Bug1 and Bug2 use a perfect **contact sensor** (with 0 range)
- We might be able to achieve better performance if we equip the robot with a range sensor that lets us look ahead (with finite range $\rho_{\max}$)
- **Range sensor:** sensor that measures the distance to an obstacle, e.g., laser range finder
- **Laser range finders** (e.g. LIDAR)
 - emit a laser beam into the environment and senses reflections from obstacles
 - essentially unidirectional, but the beam can be rotated to obtain 360 degree coverage



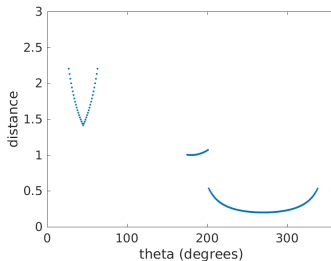
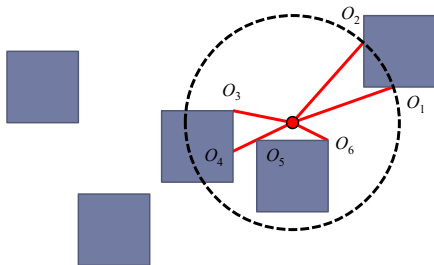
Laser range finder

- q robot/sensor position in the plane
- Assume a perfect 360 degree laser range finder with a **maximum range** $\rho_{\max}$
- **Distance measurement:** For each direction θ we measure the distance $\rho(q, \theta)$ to the first obstacle intersected by the ray from q with angle θ
- **Saturated distance:** Beyond the maximum range $\rho_{\max}$ all distance measurements are considered to be ∞



Tangent bug

- **Tangent bug:** more realistic bug algorithm that uses a range sensor to detect obstacles in advance
- The robot looks for **discontinuities** in the distance $\rho(q, \theta)$ to detect the **endpoints** of the obstacles contours within the sensing range



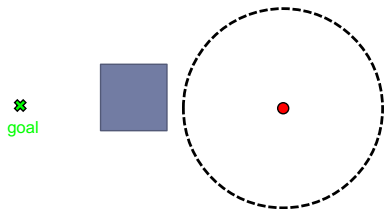
Tangent bug: behavior 1

- Behavior 1: move to goal**

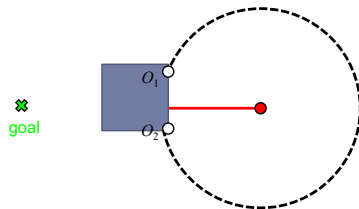
if goal is reachable move towards the goal

else move towards the discontinuity point O_k minimizing the **heuristic distance**

$$h_k(q) = d(q, O_k) + d(O_k, q_G)$$



No obstacle detected within sensing range
The robot moves towards the goal



Obstacle detected and goal not reachable
The robot moves towards O_2 because
 $h_2(q) < h_1(q)$

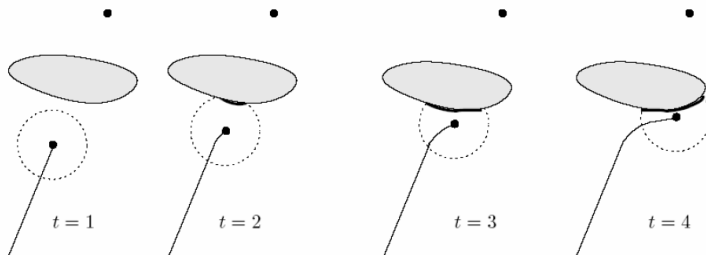
Tangent bug: behavior 1

- **Behavior 1: move to goal**

if goal is reachable move towards the goal

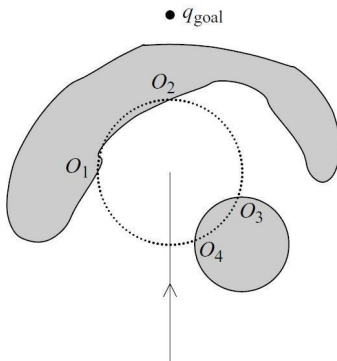
else move towards the discontinuity point O_k minimizing the **heuristic distance**

$$h_k(q) = d(q, O_k) + d(O_k, q_G)$$



Tangent bug: behavior 1

- **Behavior 1: move to goal** continues **until** goal is reached **or** the minimal heuristic distance $h_k(q)$ starts to **increase**
- Typically the minimal heuristic distance **can increase in the presence of non-convex obstacles**
- When the minimal heuristic distance increases, tangent bug switches to **Behavior 2: boundary following**

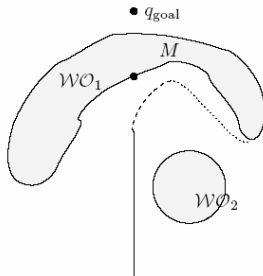


Tangent bug: behavior 2

• Behavior 2: boundary following

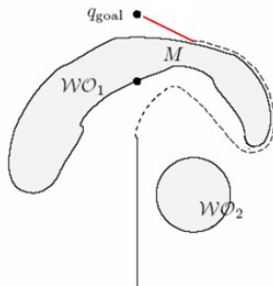
Move towards the endpoint O_k on the followed obstacle in the *chosen direction*

- **Followed obstacle:** the obstacle we're currently sensing
- While following the boundary, the robot keeps track of
 - d_{followed} shortest distance between the followed obstacle and the goal (distance between goal and point M in the figure)
 - d_{reach} shortest distance between currently blocking obstacle and goal (or distance to goal if no blocking obstacle visible)



Tangent bug: behavior 2

- **Behavior 2: boundary following** continues **until** goal is reached or d_{reach} become smaller than d_{followed}
- When $d_{\text{reach}} < d_{\text{followed}}$, tangent bug switches back to **Behavior 1: move-to-goal**



Tangent bug algorithm

1 Repeat behavior 1: move-to-goal

- a compute discontinuity points O_k
- b move to goal, if reachable, or to the discontinuity point with minimal heuristic distance $h_k(q)$

until

- a goal is reached, or
- b minimal heuristic distance $h_k(q)$ begins to increase

2 Repeat behavior 2: boundary following

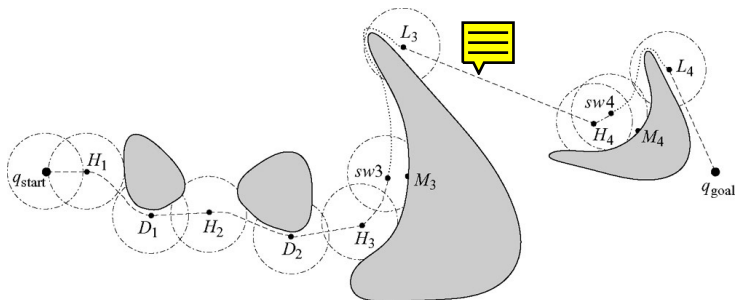
- a compute discontinuity points O_k and distances $d_{\text{reach}}, d_{\text{followed}}$
- b follow boundary continuing in same direction as before

until

- a goal is reached, or
- b a complete cycle is performed (goal is unreachable), or
- c $d_{\text{reach}} < d_{\text{followed}}$

Tangent bug

- Tangent bug is complete assuming a perfect 360 degree laser range finder
- In practice, its performance can be affected by sensor characteristics such as resolution and measurement errors



2.4 **Robot motion planning:** Artificial potential fields

Artificial potential fields

Artificial potential fields (APF): the robot is modeled as a particle under the influence of an APF $J(q)$ that superimposes

- **attractive** forces from goal
 - **repulsive** forces from obstacles
-
- *Behavior-based approach* that combines two behaviors
 - go towards the goal
 - avoid collisions with obstacles
 - Given the potential $J(q)$, the most promising direction of motion is given by the **opposite of the gradient** $-\partial J/\partial q$

Artificial potential fields

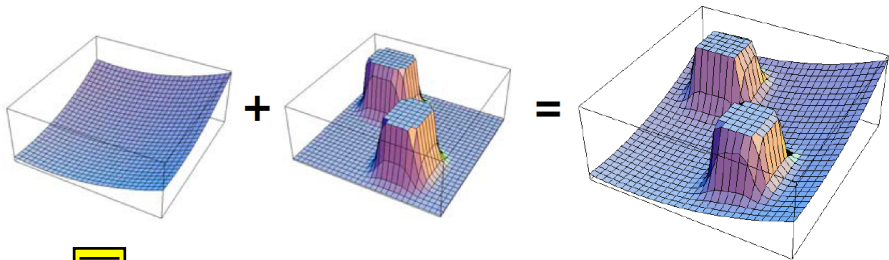
- $J_{\text{att}}(q)$ **attractive potential** (has a minimum in the goal)
- $J_{\text{rep}}(q)$ **repulsive potential** (obstacles = potential barriers)
- Artificial potential

$$J(q) = k_a J_{\text{att}}(q) + k_r J_{\text{rep}}(q)$$

with $k_a > 0$ and $k_r > 0$

- Total force acting on the robot

$$-\frac{\partial J}{\partial q} = -k_a \frac{\partial J_{\text{att}}}{\partial q} - k_r \frac{\partial J_{\text{rep}}}{\partial q}$$



Attractive potential

- Denote by q_G the goal

- Quadratic potential**

$$J_{\text{att}}(q) = \frac{1}{2} \|q - q_G\|^2$$

$$-\frac{\partial J_{\text{att}}}{\partial q} = q_G - q$$



- The attractive force of quadratic potential is large when far from goal

- Distance potential**

$$J_{\text{att}}(q) = \|q - q_G\|$$

$$-\frac{\partial J_{\text{att}}}{\partial q} = \frac{q_G - q}{\|q_G - q\|} \quad \text{for } q \neq q_G$$

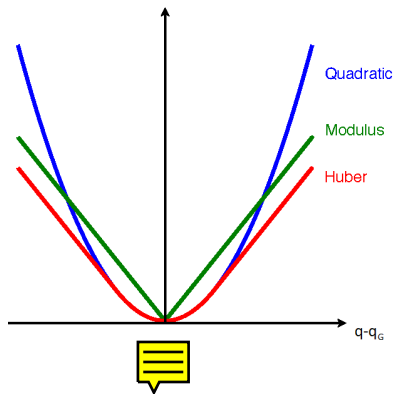


- The attractive force of distance potential has *constant modulus* but it is not defined in q_G

Attractive potential

- **Huber-like potential:** trade-off between quadratic and distance potential

$$J_{\text{att}}(q) = \begin{cases} \frac{1}{2} \|q - q_G\|^2 & \text{if } \|q - q_G\| \leq \delta \\ \delta \|q - q_G\| - \frac{1}{2} \delta^2 & \text{if } \|q - q_G\| > \delta \end{cases}$$



Repulsive potential

- For any obstacle $\mathcal{O}_k$ consider the distance between robot and obstacle

$$d(q, \mathcal{O}_k) = \min_{q' \in \mathcal{O}_k} \|q - q'\|$$

- Repulsive potential** for obstacle $\mathcal{O}_k$

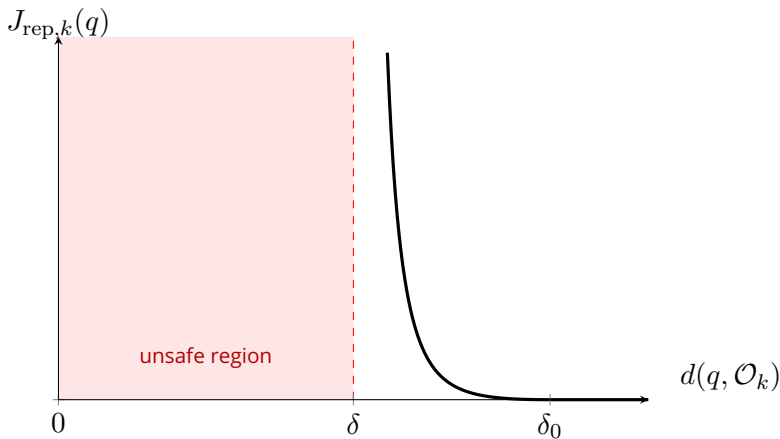
$$J_{\text{rep},k}(q) = \begin{cases} 0 & \text{if } d(q, \mathcal{O}_k) \geq \delta_0 \\ \frac{1}{\beta} \left(\frac{1}{d(q, \mathcal{O}_k) - \delta} - \frac{1}{\delta_0 - \delta} \right)^\beta & \text{if } \delta < d(q, \mathcal{O}_k) < \delta_0 \end{cases}$$

where δ **safety distance** from the obstacle

- Typical value for β is 2
- Repulsive potential**

$$J_{\text{rep}}(q) = \sum_k J_{\text{rep},k}(q)$$

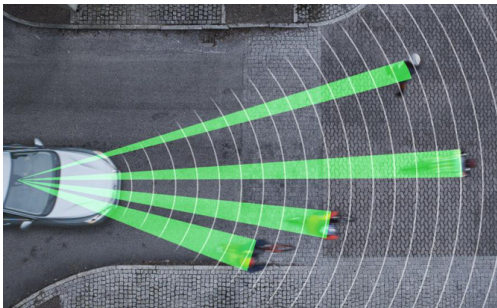
Repulsive potential



- The repulsive potential is 0 for $d(q, \mathcal{O}_k) \geq \delta_0$ where δ_0 is the **range of influence**
- The repulsive potential goes to ∞ for $d(q, \mathcal{O}_k) \rightarrow \delta$
 $\Rightarrow$ potential barrier for $d(q, \mathcal{O}_k) = \delta$

Repulsive potential

- Since the effect of the obstacle is 0 outside its range of influence only **local information** is needed
⇒ we can use sensor information instead of a map!
- In structured environments with polygonal convex obstacles, the distance $\mathcal{O}_i$ can be easily computed
- In unstructured environments or when the distance is difficult to compute
 - find the collision points at distance less than δ_0
 - define a repulsive potential for each of such collision points



Single integrator dynamics

- Consider for simplicity a robot with **single integrator** dynamics

$$\dot{q}(t) = u(t) \quad (\text{continuous time})$$

$$q(t+1) = q(t) + T_s u(t) \quad (\text{discrete time})$$

- $q(t)$ is the robot position at time t :

- $q \in \mathbb{R}^2$ for planar (2D) motion (for example, autonomous *ground* vehicles)
- $q \in \mathbb{R}^3$ for 3D motion (autonomous *air* or *underwater* vehicles)

- $u(t)$ is the velocity vector (it can be designed so as to assign to the robot a desired behaviour)

- In this model the position $q(t)$ is obtained by simply integrating the velocity $u(t)$

$$q(t) = \int_0^t u(\tau) d\tau + q(0) \quad (\text{continuous time})$$

$$q(t) = T_s \sum_{\tau=0}^{t-1} u(\tau) + q(0) \quad (\text{discrete time})$$

Policy for robots with single integrator dynamics

- For robots with single integrator dynamics the policy is easy to determine!
- Given the potential $J(q)$, the most promising direction of motion is given by the **opposite of the gradient** $-\partial J/\partial q$
- **APF-based policy:** the velocity is proportional to the force $-\partial J/\partial q$ acting on the robot

$$u(t) = -K_P \left. \frac{\partial J}{\partial q} \right|_{q=q(t)}$$

- In terms of behaviors

$$u(t) = \underbrace{-K_a \left. \frac{\partial J_{\text{att}}}{\partial q} \right|_{q=q(t)}}_{\text{go to goal}} \quad \underbrace{-K_r \left. \frac{\partial J_{\text{rep}}}{\partial q} \right|_{q=q(t)}}_{\text{avoid collision}}$$

with $K_a = K_P k_a$ and $K_r = K_P k_r$

Remark: the action $u(t)$ is a superposition (weighted sum) of two behaviors!

Feedback dynamics

- In continuous time, under the APF-based policy, the robot dynamics is a **gradient system**

$$\dot{q}(t) = -K_P \left. \frac{\partial J}{\partial q} \right|_{q=q(t)}$$

- In discrete time, under the APF-based policy, the robot dynamics is a **gradient descent**

$$q(t+1) = q(t) - \epsilon \left. \frac{\partial J}{\partial q} \right|_{q=q(t)}$$

where $\epsilon = T_s K_p$ is the **step size**

- **Gradient descent:** first-order iterative optimization algorithm for finding the minimum of a function $J(q)$
- **Gradient system:** continuous-time equivalent of gradient descent

Remark: under the APF-based policy, the robot solves an **optimization problem** by exploring the environment

Convergence of the APF method

- Assuming that the goal is outside the obstacles' range of influence, then by construction the goal q_G is a **global minimum** of the potential

$$\begin{aligned} J(q_G) &= 0 \\ J(q) &> 0 \quad \text{for } q \neq q_G \end{aligned}$$

- Assuming that
 - the stepsize is *sufficiently small*
 - the potential $J(q)$ is *sufficiently smooth* (for example, its gradient is Lipschitz continuous)

then the robot configuration $q(t)$ will converge to a **stationary point** such that

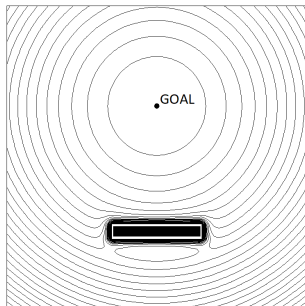
$$\frac{\partial J}{\partial q} = 0$$

Fact: when the goal q_G is the unique stationary point, the APF-based policy will lead the robot to the goal while avoiding obstacles, without requiring a priori knowledge of the map.

Local minima in the APF method

Difficulty: depending on the obstacle position and type, the APF can have **local minima**

- Local minima are due to the fact that repulsive potentials can be non-convex
- The APF method is in general **not** complete, i.e. it does not always guarantee to reach the goal even if it is possible



- If a robot enters the basin of attraction of a local minimum q_m , the robot will reach q_m and stop, i.e. the robot will be **trapped** in q_m

Local minima in the APF method

• Possible workarounds to the problem of local minima:

- 1 Define APFs without local minima
- 2 Avoid being trapped in local minima
- 3 Escape from local minima

1 Define APFs without local minima

- approaches based on *navigation functions* (instead of a repulsive potential try to navigate around the obstacles), *armonic potentials*, *dynamic programming*
- usually require **complete** knowledge of the environment: suitable for *off-line motion planning in a static environment*
- on going research topic

Local minima in the APF method

2 Avoid being trapped in local minima

9 Combine motion planning with the APF method

- *motion planning on a static map* (for example PRM) to define waypoints (subgoals)
- *APFs to navigate on-line* from one waypoint to the next one while avoiding dynamic obstacles
- **hybrid approach** combining high-level planning with low-level behaviors

10 Stochastic gradient

- add a random noise to the policy

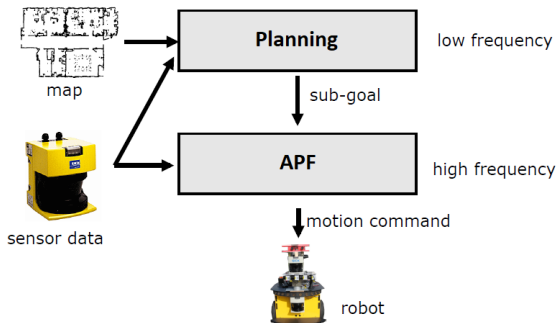
$$u(t) = -K_P \left. \frac{\partial J}{\partial q} \right|_{q=q(t)} + \eta(t)$$

where $\eta(t)$ is some stochastic process (for example white noise)

- $\eta(t)$ introduces a **random walk behavior** in the robot dynamics
- compromise between **exploitation** and **exploration**

Combine motion planning with the APF method

- Hybrid two-layered architecture for motion planning
 - A *global planner* computes paths as a sequence of waypoints (sub-goals) in a static map
 - A *local planner* (for example based on behaviors like APFs) accounts for the robot dynamic constraints and dynamic obstacles to generate feasible *collision-free* trajectories



Local minima in the APF method

5 Escape from local minima:

5a Combine motion planning with the APF method

- similar to 2a, but waypoints are redetermined when trapped in a local minimum
- for example, new waypoints can be added by motion planning to navigate around obstacles

5b Switch to other behaviors

- when trapped in a local minimum switch to another behavior such as **wall following** or **random walk**
- the new behavior is applied until outside the local minimum's basin of attraction

5c Virtual obstacles

- when trapped in a local minimum q_m add a virtual obstacle in q_m and a corresponding repulsive potential
- can work in simple cases

5d Reinforcement learning

Artificial potential fields – remarks

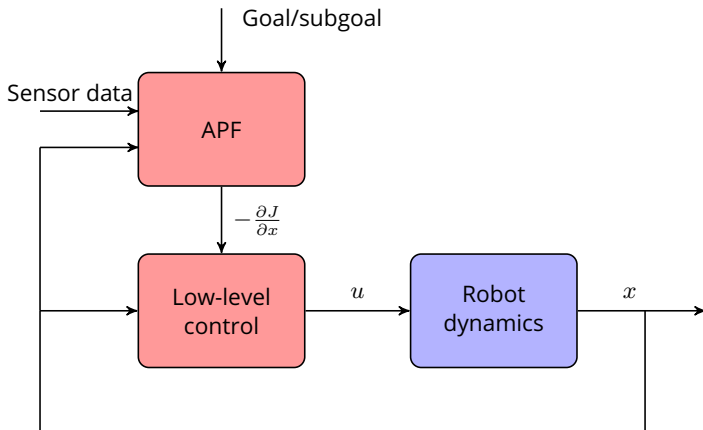
- APFs can be generalized to include **other behaviors**, e.g. **formation control** in multi-robot systems
- Other techniques for collision avoidance include *dynamic window approaches*, *nearness diagram navigation*, *vector-field histogram*, etc.
- Recall: the navigation problem is defined in the **Cartesian workspace**, but formally lives in the **configuration space**
- The APF can be defined either in the **configuration space** (general but complex) or in the **Cartesian workspace** (simpler, but requires additional control)
- In practice, unless the robot has single-integrator dynamics, the APF must be combined with a **low-level controller** to track the desired anti-gradient direction

APF for general dynamics

- For robot with general dynamics

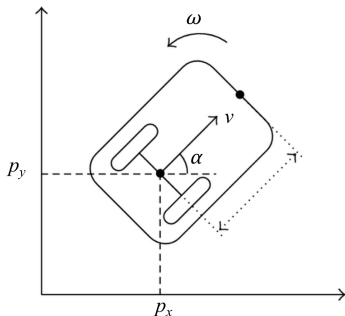
$$\dot{x}(t) = f(x(t), u(t))$$

the APF-method provides a velocity reference and a low-level controller generates $u(t)$



Example: the unicycle model

- For a real autonomous vehicle, also *orientation* is important!
- **Unicycle model:** more realistic model for planar (2D) motion



$$\dot{p}_x = \cos(\alpha) v$$

$$\dot{p}_y = \sin(\alpha) v$$

$$\dot{\alpha} = \omega$$

α orientation or heading

ω angular velocity

v forward velocity

- The robot configuration consists of position and orientation $q = (p_x, p_y, \alpha)$
- The control inputs are the angular and forward velocities $u = (\omega, v)$

Example: the unicycle model

- Suppose that we define the APF with respect to the position $p = (p_x, p_y)$ in the Cartesian space
- Give the APF $J(p)$, the most promising direction of motion is given by the opposite of the gradient
⇒ we want the velocity vector equal to

$$\dot{p}(t) = -K_P \left. \frac{\partial J}{\partial p} \right|_{p=p(t)}$$

Difficulty: For the unicycle

$$\dot{p}_x = \cos(\alpha) v$$

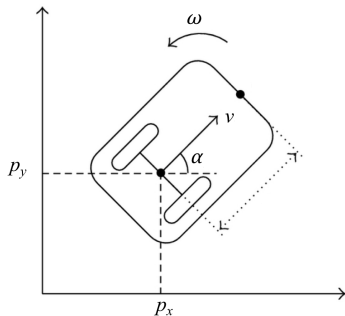
$$\dot{p}_y = \sin(\alpha) v$$

$$\dot{\alpha} = \omega$$

we cannot directly impose $\dot{p}(t) = (\dot{p}_x(t), \dot{p}_y(t))$ because also the robot orientation $\alpha(t)$ is important!

Example: the unicycle model

Idea: instead of p (that cannot be controlled), consider another point b at distance l (for example the front of the robot)



$$b_x = p_x + l \cos(\alpha)$$

$$b_y = p_y + l \sin(\alpha)$$

- Applying the chain rule

$$\begin{aligned} \dot{b}_x &= \dot{p}_x - l\dot{\alpha} \sin(\alpha) \\ \dot{b}_y &= \dot{p}_y + l\dot{\alpha} \cos(\alpha) \end{aligned} \quad \Rightarrow \quad \begin{aligned} \dot{b}_x &= v \cos(\alpha) - l\omega \sin(\alpha) \\ \dot{b}_y &= v \sin(\alpha) + l\omega \cos(\alpha) \end{aligned}$$

Example: the unicycle model

Remark: we can directly impose $\dot{b} = (\dot{b}_x, \dot{b}_y)$ by choosing v and ω

- We define the APF in the Cartesian 2D space as a function of b , i.e. $J(b)$
- Give the APF $J(b)$, the most promising direction of motion is given by the opposite of the gradient
 $\Rightarrow$ we want the velocity vector equal to

$$\dot{b}(t) = -K_P \left. \frac{\partial J}{\partial b} \right|_{b=b(t)}$$

- We can impose this direction of motion to b by inverting the mapping from (v, ω) to $\dot{b}$

$$\begin{aligned} \begin{bmatrix} v \\ \omega \end{bmatrix} &= \begin{bmatrix} \cos(\alpha) & \sin(\alpha) \\ -\sin(\alpha)/l & \cos(\alpha)/l \end{bmatrix} \begin{bmatrix} \dot{b}_x \\ \dot{b}_y \end{bmatrix} \\ &= -K_P \begin{bmatrix} \cos(\alpha) & \sin(\alpha) \\ -\sin(\alpha)/l & \cos(\alpha)/l \end{bmatrix} \left. \frac{\partial J}{\partial b} \right|_{b=b(t)} \end{aligned}$$